دانشگاه تهران
دانشکدگان فنی

برنامه‌سازی پیشرفته

راهنمای درس

مبانی برنامه‌سازی در جاوا

عبارت‌ها، حلقه‌ها، منطق بولی و گرافیک مقدماتی

پیش از شروع

این راهنما با چند مقدار و دستور کوتاه آغاز می‌شود و سپس همان ایده‌ها را در چراغ دریایی ایستگاه «نصف‌النهار» به کار می‌گیرد. قطعه‌های Java را با دنبال‌کردن مقدار متغیرها بخوانید و پیش از دیدن توضیح، نتیجه را حدس بزنید. پوستهٔ برنامه و ورودیِ نمونه در تمرین‌های مربوط آماده‌اند؛ تمرکز این بخش روی عبارت‌ها، حلقه‌ها و اثر فرمان‌های رسم است.

مرجع سریع: مقدارها و عبارت‌ها

هشت نوع پایه

هر مقدار نمونه در اعلان زیر برای نوع خودش معتبر است. پسوند `L` مقدار `long` و پسوند `f` مقدار `float` را مشخص می‌کند. نویسهٔ `char` در تک‌نقل‌قول قرار می‌گیرد.

```
byte signal = 12;
short steps = 240;
int battery = 7;
long distance = 3000000000L;
float ratio = 0.5f;
double percent = 35.0;
char marker = 'M';
boolean lampOn = true;
```

نوع را با شکل مقدار موردنیاز انتخاب کنید: `boolean` برای درست/نادرست، `char` برای یک نویسه، نوع‌های صحیح برای شمارش و نوع‌های اعشاری برای نتیجهٔ کسری به کار می‌روند. در اینجا حفظ‌کردن بازهٔ عددی نوع‌ها لازم نیست.

متغیر، مقدار اولیه و دامنه

اعلان، نام و نوع را معرفی می‌کند؛ مقداردهی اولیه نخستین مقدار را می‌دهد. `final` جلوی تغییر دوبارهٔ همان متغیر را می‌گیرد. متغیری که در آکولاد داخلی اعلان شده، بیرون آن آکولاد در دسترس نیست.

```
int remaining;           // declaration
remaining = 7;           // first value
final int capacity = 20;
{
    int used = 2;         // local to this block
    remaining -= used;   // remaining is now 5
}
// used is not available here
remaining += 3;          // 8
remaining *= 2;          // 16
remaining++;             // 17
remaining--;             // 16
```

در `remaining = 7` مقدار سمت راست به نام سمت چپ سپرده می‌شود. `+=`، `-=` و `*=` مقدار قبلی را با عمل مربوط به‌روزرسانی می‌کنند؛ `++` یک واحد می‌افزاید و `--` یک واحد می‌کاهد.

حساب و ترتیب اجرا

پنج عمل اصلی `+`، `-`، `*`، `/` و `%` هستند؛ `%` باقی‌ماندهٔ تقسیم را می‌دهد. تقسیم دو `int` بخش کسری را حذف می‌کند. برای نتیجهٔ اعشاری، پیش از تقسیم یکی از عملوندها را تبدیل کنید.

```
int sum = 7 + 2;          // 9
int difference = 7 - 2;   // 5
int product = 7 * 2;      // 14
int quotient = 7 / 2;     // 3
int remainder = 7 % 2;    // 1
double exact = (double) 7 / 2; // 3.5
```

تقدم عملگرها گروه‌بندی را تعیین می‌کند: ضرب پیش از جمع انجام می‌شود. اگر تقدم یکسان باشد، این تفریق‌ها از چپ گروه‌بندی می‌شوند. پرانتز می‌تواند گروه‌بندی را عوض کند.

```
int a = 2 + 3 * 4;        // 14: 2 + (3 * 4)
int b = 20 - 6 - 4;       // 10: (20 - 6) - 4
int c = (2 + 3) * 4;      // 20
```

گروه‌بندی با ترتیب ارزیابی عملوندها یکسان نیست. Java عملوندهای یک عبارت را از چپ به راست ارزیابی می‌کند؛ در مثال جداگانهٔ زیر، نخستین n++ مقدار 1 و دومی مقدار 2 را به جمع می‌دهد.

```
int n = 1;
int result = n++ + n++; // result = 3, n = 3
```

ردگیری باتری چراغ

از ظرفیت 20 واحد، 7 واحد مانده است. نتیجهٔ خط اول درصد واقعی نیست، چون تقسیم صحیح پیش از ضرب انجام شده است. تبدیل در خط دوم پیش از تقسیم رخ می‌دهد؛ تبدیلِ پس از تقسیم در خط سوم دیر است.

```
int remaining = 7;
int capacity = 20;
int wholeFirst = remaining / capacity * 100;        // 0
double batteryPercent = (double) remaining / capacity * 100; // 35.0
double tooLate = (double) (remaining / capacity) * 100;      // 0.0
```

پیش‌بینی

اگر remaining به 10 برسد، پیش از محاسبه بگویید هر سه خط درصد چه مقداری می‌سازند. جای تبدیل نوع را در پاسخ مشخص کنید.

واحد یک: تکرار و چراغ دریایی

تکرار با تعداد معلوم

وقتی تعداد اجراها معلوم است، for مقدار آغازین، شرط ادامه و به‌روزرسانی را کنار هم نشان می‌دهد. در این مثال بدنه دقیقاً سه بار با frame برابر 0، 1 و 2 اجرا می‌شود.

```
for (int frame = 0; frame < 3; frame++) {
    System.out.println(frame);
}
```

تکرار تا نشانهٔ پایان

در قطعهٔ بعد، محیط تمرین یک ورودی عددی آماده کرده است. 1- نشانهٔ پایان است و به جمع افزوده نمی‌شود. هر دور پس از افزودن مقدار فعلی، مقدار بعدی خوانده می‌شود؛ پس شرط دور بعد با دادهٔ تازه بررسی می‌شود.

```
// Supplied input scaffold
java.util.Scanner input = new java.util.Scanner(System.in);
int total = 0;
int reading = input.nextInt();
while (reading != -1) {
    total += reading;
    reading = input.nextInt();
}
System.out.println(total);
```

برای ورودی 7 4 -1، مقدار total به‌ترتیب 0، 4 و 11 است و همان 11 چاپ می‌شود. اگر اولین مقدار 1- باشد، بدنه حتی یک بار اجرا نمی‌شود و خروجی 0 است.

ردگیری

برای دنبالهٔ 3 2 5 -1، مقدار reading و total را درست پیش از هر بررسی شرط بنویسید. سپس بگویید چرا نشانهٔ پایان وارد جمع نمی‌شود.

رسم و حرکت در پوستهٔ آماده

این پوستهٔ ACM برای مثال فراهم شده است. چهار فرمان GRect، GOval، GLine و GLabel به‌ترتیب یک مستطیل، بیضی، خط و نوشته می‌سازند. مختصات مستطیل و بیضی همراه عرض و ارتفاع است؛ خط با دو سر مشخص می‌شود و جای نوشته نقطهٔ خط مبنای آن است. هر شکل با add روی بوم دیده می‌شود.

```
import acm.graphics.*;
import acm.program.GraphicsProgram;

// Supplied ACM program shell
public class LighthouseDemo extends GraphicsProgram {
    public void run() {
        GRect tower = new GRect(50, 90, 34, 110);
        GOval light = new GOval(54, 76, 26, 18);
        GLine shore = new GLine(0, 210, 300, 210);
        GLabel sign = new GLabel("Meridian", 24, 70);
        add(tower);
        add(light);
        add(shore);
        add(sign);

        GOval boat = new GOval(30, 190, 24, 12);
        add(boat);
        for (int frame = 0; frame < 40; frame++) {
            boat.move(2, 0);
            pause(40);
        }
    }
}
```

در هر قاب، move(2, 0) بیضی قایق را دو واحد به راست می‌برد و pause(40) مکث کوتاهی پیش از قاب بعدی ایجاد می‌کند. به تغییر یک قاب فکر کنید: اگر آرگومان اول move منفی شود، مسیر دیده‌شده چه تغییری می‌کند؟

مرجع سریع: مقایسه و تصمیم

رابطه‌ها و منطق بولی

نتیجهٔ هر مقایسه از نوع boolean است. == برابری و != نابرابری را می‌سنجند؛ >، <، >= و <= نیز دو مقدار را از نظر ترتیب مقایسه می‌کنند. برای مقایسه از == استفاده کنید؛ = مقدار را به متغیر می‌سپارد.

```
int boats = 3;
boolean crowded = boats > 2;    // true
boolean empty = boats == 0;     // false
boolean room = boats <= 4;      // true
boolean changed = boats != 3;   // false
boolean lower = boats < 4;      // true
boolean atLeast = boats >= 3;   // true
```

&& وقتی درست است که هر دو شرط درست باشند؛ || وقتی درست است که دست‌کم یکی درست باشد. ! نتیجهٔ یک شرط را وارونه می‌کند. Java عملوند چپ && و || را اول می‌سنجد و گاهی اصلاً به عملوند راست نمی‌رسد.
در دو قطعهٔ مستقل زیر، boats می‌تواند صفر باشد. در &&، اگر شرط چپ نادرست باشد، تقسیم سمت راست اجرا نمی‌شود. در ||، اگر شرط چپ درست باشد، تقسیم سمت راست اجرا نمی‌شود. وقتی boats صفر نیست، هر دو قطعه به سنجش میانگین می‌رسند.

```
int boats = 0;
int total = 18;
int limit = 5;
boolean overLimit = boats != 0 && total / boats > limit; // false
```

```
int boats = 0;
int total = 18;
int limit = 5;
boolean emptyOrOverLimit = boats == 0 || total / boats > limit; // true
```

اگر تقسیم را پیش از نگهبانِ صفر بگذارید، هنگام صفر بودن boats خطای تقسیم بر صفر رخ می‌دهد. & و | نیز هر دو طرف عبارت بولی را ارزیابی می‌کنند و در اینجا جای && و || را نمی‌گیرند. تقدم و هم‌جهتی عملگرها گروه‌بندی عبارت را مشخص می‌کنند؛ ترتیب ارزیابی عملوندها جدا از آن است و در Java از چپ به راست پیش می‌رود. برای روشن‌بودن گروه‌بندی شرط‌های ترکیبی، پرانتز بگذارید.

پیش‌بینی

اگر boats برابر 3 باشد، مقدار هر دو متغیر بولی بالا را پیش‌بینی کنید. سپس بگویید کدام بخش‌ها این بار ارزیابی می‌شوند.

انتخاب مسیر

چراغ وضعیت بندر در پوستهٔ آماده، با مقدار wind تصمیم می‌گیرد. شرط‌ها به‌ترتیب بررسی می‌شوند و فقط نخستین شاخهٔ درست اجرا می‌شود. else برای حالتی است که هیچ شرط پیشین درست نباشد.

```
int wind = 18;
if (wind >= 30) {
    System.out.println("Closed");
} else if (wind >= 15) {
    System.out.println("Proceed carefully");
} else {
    System.out.println("Open");
}
```

برای انتخاب کوتاه میان دو مقدار، عبارت شرطی ?: همان تصمیم را در یک مقدار می‌سازد. در مثال زیر فقط یکی از دو رشته انتخاب می‌شود.

```
int boats = 3;
String message = boats == 0 ? "Empty dock" : "Boat present";
System.out.println(message);
```

وقتی یک مقدار مشخص چند حالت جدا دارد، switch آن را با caseها می‌سنجد. break پس از اجرای شاخه، خروج از switch را تضمین می‌کند؛ بدون آن، اجرا ممکن است به case بعدی ادامه یابد. default حالت‌های نام‌برده‌نشده را می‌گیرد.

```
int signal = 1;
switch (signal) {
    case 0:
        System.out.println("Closed");
        break;
    case 1:
        System.out.println("Caution");
        break;
    default:
        System.out.println("Unknown");
        break;
}
```

واحد دو: تختهٔ بندر

یک شمارش، دو نوع حلقه

این دو قطعهٔ مستقل برای `n = 4` عددهای 0 تا 3 را چاپ می‌کنند. در هر دو، مقدار آغازین فقط یک‌بار ساخته می‌شود؛ پیش از هر دور شرط سنجیده می‌شود؛ پس از بدنه، شمارنده یک واحد زیاد می‌شود. در `while` این سه جایگاه در خط‌های جدا دیده می‌شوند.

```
int n = 4;
for (int i = 0; i < n; i++) {
    System.out.println(i);
}
```

```
int n = 4;
int i = 0;                  // initialization
while (i < n) {             // condition
    System.out.println(i);  // body
    i++;                    // update
}
```

برای `n = 1`، هر دو فقط 0 را چاپ می‌کنند: پس از به‌روزرسانی، شرط `1 < 1` نادرست است. اگر `i++` را از `while` بردارید، مقدار `i` تغییر نمی‌کند و شرط پایان نمی‌رسد.

تختهٔ شطرنجی متنی

برای تخته‌ای با `rows` سطر و `cols` ستون، حلقهٔ بیرونی هر سطر و حلقهٔ درونی هر ستون همان سطر را می‌سازد. شرط‌های `row < rows` و `col < cols` از خانهٔ بیرونِ تخته جلوگیری می‌کنند. خانهٔ بالا و چپ، یعنی `row = 0` و `col = 0`، با `(row + col) % 2 == 0` علامت X می‌گیرد؛ خانهٔ کناری علامت `.` دارد.

```
int rows = 4;
int cols = 4;
for (int row = 0; row < rows; row++) {
    for (int col = 0; col < cols; col++) {
        System.out.print((row + col) % 2 == 0 ? "X " : ". ");
    }
    System.out.println();
}
```

```
X . X .
. X . X
X . X .
. X . X
```

برای ساخت این تخته فقط خروجی متنی Java لازم است. سطر اول چهار خانه دارد؛ در سطر دوم، چون `row` یک واحد زیاد شده است، جای `X` و `.` عوض می‌شود. اگر `cols` را به 3 تغییر دهید، تعداد سطرها ثابت می‌ماند و هر سطر سه خانه خواهد داشت.

تلاش کنید

برای تختهٔ 3 در 5، خروجی را پیش از اجرا روی کاغذ بنویسید. مقدار `row + col` را برای خانهٔ بالا و چپ و دو همسایهٔ آن نیز پیدا کنید.

پیوند اختیاری با رسم

در پوستهٔ آمادهٔ ACM می‌توان همان دو حلقه و شرط زوج‌بودن را نگه داشت و تنها برای خانه‌های X یک مستطیل کشید. `col * cell` جای افقی و `row * cell` جای عمودی را می‌دهد؛ `setFilled(true)` درون مستطیل را پر و `add(tile)` آن را روی بوم نمایان می‌کند. این قطعه در بخش آمادهٔ `run` قرار می‌گیرد.

```
int rows = 4;
int cols = 4;
int cell = 18;
for (int row = 0; row < rows; row++) {
    for (int col = 0; col < cols; col++) {
        if ((row + col) % 2 == 0) {
            GRect tile = new GRect(col * cell, row * cell, cell, cell);
            tile.setFilled(true);
            add(tile);
        }
    }
}
```

مرور پایانی

پیش از رفتن به مبحث بعدی، پاسخ هر پرسش را با یک ردگیری کوتاه برای خود توضیح دهید:

۱. چرا `7 / 2` و `7 / 2 (double)` یک نتیجه ندارند؟
۲. برای `n = 4`، ترتیب مقدارهای `i` و زمان بررسی شرط در دو حلقهٔ هم‌ارز چیست؟
۳. اگر `boats = 0` باشد، هر یک از دو شرطِ نگهبانِ تقسیم چگونه تصمیم می‌گیرد؟
۴. در تختهٔ 4 در 4، خانهٔ سطر 1 و ستون 1 چه علامتی دارد و کدام فرمان ACM همان خانه را رسم می‌کند؟